LAPORAN PRAKTIKUM STRUKTUR DATA ADT

IMPLEMENTASI STACK DAN PRINSIP LIFO DALAM JAVA

Disusun Oleh:

Ihsanul Zaky EL-Muhammady

NIM:2511533001

Dosen Pengampu: Dr.Wahyudi S.T , M.T

Asisten Laboratorium: Zahran Azaria Anvaya

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG 2026

KATA PENGANTAR

Dengan penuh rasa syukur, saya mengucapkan terima kasih kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum “Implementasi Stack dan Prinsip LIFO dalam Java” ini dapat diselesaikan dengan baik.

Saya juga menyampaikan penghargaan yang sebesar-besarnya kepada Dr. Wahyudi, S.T., M.T. selaku dosen pengampu Mata Kuliah Struktur Data, yang telah memberikan arahan, bimbingan, serta motivasi selama proses perkuliahan dan praktikum. Ucapan terima kasih juga saya sampaikan kepada asisten laboratorium yang telah membantu dan membimbing jalannya praktikum dengan sabar dan penuh perhatian.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Stack dengan prinsip LIFO (Last In, First Out), serta penerapannya dalam bahasa Java, baik menggunakan library bawaan, ArrayList, maupun array statis. Selain itu, praktikum ini juga membahas penerapan Stack dalam kasus sederhana seperti evaluasi notasi postfix dan simulasi riwayat browser.

Saya menyadari bahwa laporan ini masih memiliki banyak kekurangan. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di kemudian hari.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR .....

DAFTAR ISI .....

BAB I PENDAHULUAN .....

1.1 Latar Belakang .....

1.2 Tujuan .....

1.3 Manfaat .....

BAB II PEMBAHASAN .....

2.1 Devinisi Stack dalam Java .....

2.2 Langkah pengerjaan .....

2.3 Latihan .....

BAB III KESIMPULAN .....

3.1 Kesimpulan .....

3.2 Saran .....

DAFTAR PUSTAKA .....

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengolahan data, tidak semua kondisi membutuhkan akses yang bebas terhadap setiap elemen. Ada situasi tertentu yang justru memerlukan aturan khusus, di mana data yang terakhir dimasukkan akan diproses lebih dulu. Konsep ini dikenal dengan LIFO (Last In, First Out) dan diterapkan dalam struktur data Stack atau tumpukan.

Jika dibandingkan dengan ArrayList yang bisa diakses menggunakan indeks secara fleksibel, Stack memiliki aturan yang lebih terbatas karena hanya beroperasi pada satu bagian saja, yaitu bagian atas (top). Meskipun begitu, cara kerja ini sangat berguna dalam berbagai kasus seperti fitur undo, navigasi kembali, perhitungan ekspresi matematika, serta proses pemanggilan fungsi dalam program. Pada praktikum ini, Stack akan diimplementasikan dengan beberapa cara, mulai dari menggunakan library Java, ArrayList, hingga array statis, lalu diterapkan pada contoh kasus sederhana.

1.2 Tujuan

1. Memahami konsep dasar Stack dan cara kerja LIFO.
2. Menggunakan Stack dengan memanfaatkan class bawaan Java.
3. Membuat Stack secara manual menggunakan ArrayList.
4. Mengimplementasikan Stack menggunakan array statis.
5. Menerapkan Stack pada permasalahan seperti notasi postfix dan riwayat browser.
6. Mengetahui cara kerja serta efisiensi operasi dasar pada Stack.

1.3 Manfaat

1. Dapat memahami perbedaan Stack dengan struktur data lain seperti Array dan ArrayList.
2. Mengetahui pentingnya aturan akses data dalam suatu sistem.
3. Mampu mengimplementasikan konsep Stack ke dalam program sederhana.
4. Melatih logika dalam menyelesaikan masalah menggunakan Stack.
5. Menambah pengalaman dalam membuat dan menguji program.
6. Melatih kemampuan dalam menemukan dan memperbaiki kesalahan pada program.

BAB II

PEMBAHASAN

2.1 Konsep Stack dalam Java

Stack merupakan salah satu struktur data yang menyimpan elemen secara berurutan dengan prinsip LIFO (Last In, First Out), yaitu data yang terakhir dimasukkan akan menjadi yang pertama dikeluarkan. Pada Stack, proses penambahan dan penghapusan data hanya bisa dilakukan pada satu sisi saja, yaitu bagian atas yang disebut top.

Beberapa operasi dasar yang ada pada Stack antara lain:

- push(e): digunakan untuk menambahkan elemen ke bagian paling atas Stack.
- pop(): berfungsi untuk mengambil sekaligus menghapus elemen yang berada di posisi teratas.
- peek() / top(): digunakan untuk melihat elemen teratas tanpa menghapusnya.
- isEmpty(): untuk mengecek apakah Stack dalam keadaan kosong atau tidak.
- size(): untuk mengetahui jumlah elemen yang ada di dalam Stack.

Dalam bahasa Java, Stack bisa dibuat dengan beberapa cara. Cara paling mudah adalah menggunakan class bawaan seperti java.util.Stack, walaupun class ini termasuk versi lama (legacy) dan cenderung lebih lambat karena bersifat thread-safe. Selain itu, Stack juga bisa dibuat secara manual menggunakan array statis atau bantuan struktur lain seperti ArrayList maupun LinkedList, agar lebih memahami cara kerja Stack secara internal.

2.2 Langkah pengerjaan

1. NilaiMaksimum_2511533001

```java
package pekan3_2511533001;

import java.util.Stack;

public class NilaiMaksimum_2511533001 {
    public static int max_3001(Stack<Integer> s_3001) {
        Stack<Integer> backup_3001 = new Stack<Integer>();
        int maxValue_3001 = s_3001.pop();
        backup_3001.push(maxValue_3001);
        while (!s_3001.isEmpty()) {
            int next_3001 = s_3001.pop();
            backup_3001.push(next_3001);
            maxValue_3001 = Math.max(maxValue_3001, next_3001);
        }
        while (!backup_3001.isEmpty()) {
            s_3001.push(backup_3001.pop());
        }
        return maxValue_3001;

    }
    public static void main(String[] args) {
        Stack<Integer> s_3001 = new Stack<Integer>();
        s_3001.push(70);
        s_3001.push(12);
        s_3001.push(20);
        System.out.println("isi stack "+s_3001);
        System.out.println("Stack Teratas "+s_3001.peek());
        System.out.println("NIlai maksimum "+max_3001(s_3001));
    }

}
```

Gambar 2.1

Penjelasan program:

1. Stack<Integer> backup_3001 = new Stack<>(); → Membuat stack tambahan yang digunakan untuk menyimpan elemen sementara, sehingga isi stack utama tidak berubah setelah proses selesai.

2. int maxValue_3001 = s_3001.pop(); → Mengambil elemen teratas sebagai nilai awal untuk perbandingan dalam mencari nilai maksimum.

3. while (!s_3001.isEmpty()) { ... } → Perulangan dilakukan sampai stack utama kosong. Setiap elemen diambil (pop), lalu dibandingkan dengan nilai maksimum menggunakan Math.max(), kemudian disimpan ke dalam backup_3001.

4. while (!backup_3001.isEmpty()) { s_3001.push(backup_3001.pop()); } → Setelah semua elemen diproses, isi stack dikembalikan lagi ke stack utama agar susunannya tetap seperti semula.

5. Analisis Kompleksitas: Waktu O(n) karena setiap elemen diproses dua kali (dipindahkan ke stack bantu lalu dikembalikan lagi). Ruang O(n) karena membutuhkan stack tambahan untuk penyimpanan sementara.

Output:

```
<terminated> NilaiMaksimum_2511533001 [
isi stack [70, 12, 20]
Stack Teratas 20
NIlai maksimum 70
```

Gambar 2.2

2. SiswaStack_2511533001

```java
package pekan3_2511533001;
import java.util.ArrayList;
class Siswa_3001 {
    String nama_3001;
    int nim_3001;

public Siswa_3001(String nama, int nim) {
    this.nama_3001 = nama;
    this.nim_3001 = nim;
}


    public String toString() {
        return "Nim: " + nim_3001 + ", Nama: " + nama_3001;
    }
}

public class SiswaStack_2511533001 {
    private ArrayList<Siswa_3001> stack;

    public SiswaStack_2511533001 () {
        stack = new ArrayList<>();
    }

    public void push(Siswa_3001 mhs) {
        stack.add(mhs);
    }

    public Siswa_3001 pop_3001 () {
        if (!isEmpty()) {
```

```java
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswa_3001 peek_3001() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }
    public void tampilkansiswa_3001() {
        for (int i = stack.size () - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }
public static void main(String[] args) {
     SiswaStack_2511533001 studentStack_3001 = new SiswaStack_2511533001 ();

     Siswa_3001 mhs1 = new Siswa_3001("Ali", 1);
     Siswa_3001 mhs2 = new Siswa_3001("Boby", 2);
     Siswa_3001 mhs3 = new Siswa_3001("Charles", 3);

     studentStack_3001.push(mhs1);
     studentStack_3001.push(mhs2);
     studentStack_3001.push(mhs3);

    System.out.println("Siswa di dalam stack:");
    studentStack_3001.tampilkansiswa_3001();

    System.out.println("siswa teratas " + studentStack_3001.peek_3001());
    System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack_3001.pop_3001());
    System.out.println("daftar siswa setelah di pop :");
    studentStack_3001.tampilkansiswa_3001();
}
}
```

Gambar 2.3

Penjelasan program:

1. private ArrayList<Siswa_2511533001> stack_3001; → Menggunakan ArrayList sebagai media penyimpanan data, tetapi aksesnya dibatasi hanya pada elemen terakhir (size()-1) agar sesuai dengan konsep Stack.

2. push_3001() memanggil stack_3001.add(mhs); → Digunakan untuk menambahkan data ke bagian paling akhir yang dianggap sebagai posisi top.

3. pop_3001() memanggil stack_3001.remove(size()-1); → Berfungsi untuk mengambil sekaligus menghapus elemen terakhir dari stack.

4. peek_3001() memanggil stack_3001.get(size()-1); → Digunakan untuk melihat elemen paling atas tanpa menghapusnya dari stack.

5. tampilkanSiswa() melakukan perulangan dari belakang ke depan (i >= 0) supaya tampilan data sesuai dengan konsep stack, di mana elemen teratas ditampilkan lebih dulu.

6. Program ini menunjukkan konsep ADT (Abstract Data Type), di mana operasi seperti push, pop, dan peek digunakan sebagai antarmuka, sementara proses penyimpanan sebenarnya diatur oleh ArrayList di dalamnya.

Output:

```
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 2, Nama: Boby
daftar siswa setelah di pop :
Nim: 1, Nama: Ali
```

Gambar 2.4

3. StackArray_2511533001 & StackArrayDriver_2511533001

```java
package pekan3_2511533001;

public class stackArray_2511533001 {
    static final int MAX_3001 = 1000;
    int top_3001;
    int a_3001[] = new int [MAX_3001];
    boolean isEmpty_3001()
    {
            return (top_3001 < 0);
    }
    stackArray_2511533001()
    {
            top_3001 = -1;
    }
    boolean push_3001(int x)
    {
        if (top_3001 >= (MAX_3001 - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a_3001[++top_3001] = x;
            System.out.println(x + "dimasukkan dalam stack");
            return true;
        }
    }
    int pop_3001()
    {
        if (top_3001 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a_3001 [top_3001--];
            return x;
        }
    }
    int peek_3001()
    {
        if (top_3001 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a_3001 [top_3001];
            return x;
        }
    }
    void print_3001() {
        for (int i = top_3001;i>-1;i--) {
            System.out.println(" "+ a_3001 [i]);
        }
    }
}
```

```java
package pekan3_2511533001;

public class stackArraayDriver_2511533001 {

    public static void main(String[] args) {
        stackArray_2511533001 s = new stackArray_2511533001();
        s.push_3001(10);
        s.push_3001(20);
        s.push_3001(30);
        System.out.println(s.pop_3001() + " dikeluarkan dari srack");
        System.out.println("Elemen teratas adalah :" + s.peek_3001());
        System.out.println("Elemen pada stack :");
        s.print_3001();
    }
}
```

Gambar 2.5

Penjelasan program:

1. static final int MAX_3001 = 1000; dan int a_3001[]; → Program menggunakan array statis dengan ukuran tetap sebagai tempat penyimpanan data pada stack.
2. int top_3001 = -1; → Variabel ini berfungsi sebagai penunjuk posisi teratas. Nilai awal -1 menandakan bahwa stack masih kosong.
3. push_3001(int x): → Method untuk menambahkan elemen ke dalam stack.
4. Dilakukan pengecekan if (top_3001 >= MAX_3001 - 1) → Untuk memastikan stack tidak melebihi kapasitas (mencegah overflow).
5. a_3001[++top_3001] = x; → Nilai top dinaikkan terlebih dahulu, kemudian elemen dimasukkan ke posisi tersebut.
6. pop_3001(): → Method untuk mengambil sekaligus menghapus elemen teratas.
7. Dilakukan pengecekan if (top_3001 < 0) → Untuk memastikan tidak terjadi pengambilan data saat stack kosong (underflow).
8. return a_3001[top_3001--]; → Mengembalikan nilai elemen teratas, lalu posisi top diturunkan.
9. Pada bagian driver (stackArrayDriver_3001), dilakukan pengujian konsep LIFO dengan memasukkan data 10, 20, 30. Saat dilakukan pop, nilai yang keluar adalah 30, dan saat peek ditampilkan nilai 20.

Output:

```
10dimasukkan dalam stack
20dimasukkan dalam stack
30dimasukkan dalam stack
30 dikeluarkan dari srack
Elemen teratas adalah :20
Elemen pada stack :
 20
 10
```

Gambar 2.6

4.StackPostfix_2511533001

```java
package pekan3_2511533001;
import java.util.Scanner;
public class StackPostfix_2511533001 {
    public static int postfixEvaluate(String expression) {
        Stack<Integer> s_3001 = new Stack<Integer>();
        Scanner input = new Scanner(expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) {   // an operand (integer)
                s_3001.push (input.nextInt());
            } else {
                String operator = input.next();
                int operand2_3001 = s_3001.pop();
                int operand1_3001 = s_3001.pop();
                if (operator.equals("+")) {
                    s_3001.push (operand1_3001 + operand2_3001);
                }else if (operator.equals("-")) {
                    s_3001.push (operand1_3001 - operand2_3001);
                }else if (operator.equals("*")) {
                    s_3001.push (operand1_3001 * operand2_3001);
                }else {
                    s_3001.push (operand1_3001 / operand2_3001);
                }
            }
        }
        input.close();
        return s_3001.pop();
    }
    public static void main (String[] args) {
        System.out.println("hasil postfix= "+postfixEvaluate ("5 2 4 * + 7 -"));
    }
}
```

Gambar 2.7

Penjelasan program:

1. Scanner input_3001 = new Scanner(expression); → Digunakan untuk membaca dan memisahkan setiap token dari ekspresi berdasarkan spasi.

2. if (input_3001.hasNextInt()) s_3001.push(...); → Jika token berupa angka, maka langsung dimasukkan ke dalam stack.

3. else { ... } → Jika token adalah operator seperti (+, -, *, /):

- operand2_3001 = s_3001.pop(); → Mengambil nilai operand kedua (kanan) terlebih dahulu.
- operand1_3001 = s_3001.pop(); → Kemudian mengambil operand pertama (kiri).
- Setelah itu dilakukan operasi sesuai operator, lalu hasilnya dimasukkan kembali ke dalam stack.

4. Proses Ekspresi "5 2 4 * + 7 -":

1. Push: 5, 2, 4

2. * → ambil 4 dan 2 → 2×4 = 8 → push 8

3. + → ambil 8 dan 5 → 5+8 = 13 → push 13

4. Push: 7

5. - → ambil 7 dan 13 → 13−7 = 6 → push 6

5. return s_3001.pop(); → Mengambil hasil akhir dari stack, yaitu 6.

Output:

```
hasil postfix= 6
```

Gambar 2.8

2.3 Latihan

Deskripsi Program:

Program mensimulasikan fitur navigasi browser menggunakan prinsip LIFO. Setiap halaman yang dikunjungi disimpan di stack, dan tombol "Back" menghapus halaman teratas

1.Program Website_2511533001

```java
package pekan3_2511533001;

public class Website_2511533001 {
    private String judul_3001;
    private String url_3001;

    public Website_2511533001(String judul_3001, String url_3001) {
        this.judul_3001 = judul_3001;
        this.url_3001 = url_3001;
    }

    public String getJudul_3001() {
        return judul_3001;
    }

    public String getUrl_3001() {
        return url_3001;
    }

    public void setJudul_3001(String judul_3001) {
        this.judul_3001 = judul_3001;
    }

    public void setUrl_3001(String url_3001) {
        this.url_3001 = url_3001;
    }

    @Override
    public String toString() {
        return "Judul: " + judul_3001 + "\nURL : " + url_3001;
    }
```

Gambar 2.9

Deskripsi program Website_2511533001 (ADT Data):
1. Kelas ini memiliki dua atribut utama, yaitu judul_3001 dan url_3001, yang disimpan dengan akses private agar data lebih terkontrol.
2. Untuk mengakses dan mengubah nilai atribut tersebut, disediakan method getter dan setter.
3.Selain itu, terdapat method toString() yang digunakan untuk menampilkan informasi judul dan URL dalam format yang lebih rapi dan mudah dibaca.

2.Program Browser_2511533001

```java
package pekan3_2511533001;
import java.util.Scanner;
import java.util.Stack;
public class Browser_2511533001 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Stack<Website_2511533001> history_3001 = new Stack<>();
        int pilihan;
        do {
            System.out.println("\n=== Browser History NIM: 2511533001 ===");
            System.out.println("1. Kunjungi Website");
            System.out.println("2. Tombol Back");
            System.out.println("3. Lihat Halaman Aktif");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
            System.out.print("Pilih: ");
            pilihan = scanner.nextInt();
            scanner.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan Judul: ");
                    String judul_3001 = scanner.nextLine();
                    System.out.print("Masukkan URL: ");
                    String url_3001 = scanner.nextLine();

                    history_3001.push(new Website_2511533001(judul_3001, url_3001));
                    System.out.println("Berhasil mengunjungi halaman.");
                    break;

                case 2:
```

```java
                    if (history_3001.isEmpty()) {
                        System.out.println("History kosong. Tidak ada halaman untuk dikembalikan.");
                    } else {
                        Website_2511533001 lastSite = history_3001.pop();
                        System.out.println("Berhasil kembali. Halaman yang dihapus: " + lastSite.get
                    }
                    break;

                case 3:
                    if (history_3001.isEmpty()) {
                        System.out.println("History kosong. Tidak ada halaman aktif.");
                    } else {
                        Website_2511533001 currentSite = history_3001.peek();
                        System.out.println("\n=== Halaman Aktif ===");
                        System.out.println(currentSite.toString());
                    }
                    break;

                case 4:
                    System.out.println("Status History: " + (history_3001.isEmpty() ? "Kosong" : "Te
                    System.out.println("Jumlah total riwayat tersimpan: " + history_3001.size());
                    break;

                case 5:
                    System.out.println("Keluar dari program...");
                    break;

                default:
                    System.out.println("Pilihan tidak valid.");
                    break;
            }

        } while (pilihan != 5);

        scanner.close();
    }
}
```

Gambar 2.10

Deskripsi program Browser_2511533001 (Logika Aplikasi)

1 Stack<Website_2511533001> history_3001 → Digunakan sebagai struktur data utama untuk menyimpan riwayat halaman yang dikunjungi.

2. case 1 (Kunjungi Website): Membuat objek Website baru, lalu dimasukkan ke dalam stack menggunakan push(). Halaman yang baru ditambahkan otomatis berada di posisi paling atas (top).

3. case 2 (Tombol Back): Dilakukan pengecekan dengan isEmpty(). Jika stack tidak kosong, maka pop() digunakan untuk menghapus halaman terakhir, kemudian ditampilkan pesan konfirmasi. Jika kosong, akan muncul pesan bahwa tidak ada data.

4. case 3 (Lihat Halaman Aktif): Menggunakan peek() untuk melihat halaman yang berada di posisi teratas tanpa menghapusnya dari stack.

5. case 4 (Cek Status): Menampilkan kondisi stack (kosong atau tidak) serta jumlah data yang tersimpan dengan history_3001.size().

6. do-while dan switch: Digunakan untuk membuat program berjalan secara interaktif di konsol, sehingga pengguna bisa terus memilih menu sampai memilih opsi keluar.

Output:

```
=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih: 1
Masukkan Judul: ILEARN UNAND
Masukkan URL: https://it.ilearn.unand.ac.id/
Berhasil mengunjungi halaman.

=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih: 3

=== Halaman Aktif ===
Judul: ILEARN UNAND
URL : https://it.ilearn.unand.ac.id/

=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
```

```
Pilih: 4
Status History: Terdapat Data
Jumlah total riwayat tersimpan: 1

=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih: 2
Berhasil kembali. Halaman yang dihapus: ILEARN UNAND

=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih: 4
Status History: Kosong
Jumlah total riwayat tersimpan: 0

=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar

=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih: 2
Berhasil kembali. Halaman yang dihapus: ILEARN UNAND

=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih: 4
Status History: Kosong
Jumlah total riwayat tersimpan: 0

=== Browser History NIM: 2511533001 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilih: 5
Keluar dari program...
```

Gambar 2.11

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum implementasi Stack dalam Java, dapat disimpulkan bahwa Stack adalah struktur data linier yang menggunakan prinsip LIFO (Last In, First Out), di mana semua operasi dilakukan pada bagian atas (top). Stack dapat dibuat dengan beberapa cara, seperti menggunakan class bawaan java.util.Stack, memanfaatkan ArrayList, maupun menggunakan array statis.

Operasi dasar seperti push, pop, peek, dan isEmpty memiliki waktu eksekusi yang cepat (O(1)), sehingga cocok digunakan pada kasus tertentu yang tidak membutuhkan akses acak. Penerapan Stack pada praktikum ini, seperti evaluasi notasi postfix dan simulasi riwayat browser, menunjukkan bahwa konsep ini banyak digunakan dalam berbagai aplikasi, termasuk sistem program dan fitur aplikasi sehari-hari. Selain itu, penting juga untuk memperhatikan kondisi seperti overflow dan underflow agar program tetap berjalan dengan baik.

3.2 Saran

Sebaiknya dosen menyelenggarakan sesi pra-praktikum di kelas sebagai pengantar materi, agar mahasiswa memperoleh pemahaman awal yang lebih baik. Dengan demikian, potensi kepanikan atau kesalahan selama praktikum dapat diminimalkan. Disarankan agar materi praktikum dibagikan terlebih dahulu melalui platform iLearn, sehingga mahasiswa memiliki waktu yang cukup untuk mempelajari dan mempersiapkan diri sebelum pelaksanaan praktikum.

DAFTAR PUSTAKA

[1] Oracle. "Stack (Java Platform SE 17)". The Java™ Tutorials. 2024.

Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/u til/Stack.html